

Cloud Based IaaS Virtual Infrastructure Provisioning and Auto Scaling System

A CAPSTONE PROJECT REPORT

Submitted in the partial fulfilment for the Course of

CSA1522- CLOUD COMPUTING AND BIG DATA ANALYTICS

to the award of the degree of

BACHELOR OF ENGINEERING

IN

B.TECH(AI&ML)

Submitted by

R. Pranathi(192525076)

M. Meghana(192525435)

M. Divyasree(192524098)

Under the Supervision of

Dr. P. Rajaram



Saveetha Institute of Medical and Technical Sciences

Chennai-602105

September 2026



SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai-602105



DECLARATION

We, **R.Pranathi, M.Meghana, M.Divyasree** of the **B.tech (AI&ML)** Saveetha Institute of Medical and Technical Sciences, Saveetha University, Chennai, hereby declare that the Capstone Project Work entitled “**Cloud Based IaaS Virtual Infrastructure Provisioning and Auto Scaling System**” is the result of our own Bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with principles of engineering ethics.

Place: Chennai

Date:

Signature of the Students with Names

R. Pranathi(192525076)

M. Meghana(192525435)

M. Divyasree(192524098)



SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105



BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “**Cloud Based IaaS Virtual Infrastructure Provisioning and Auto Scaling System**” has been carried out by **R. Pranathi, M. Meghana, M. Divyasree** under the supervision of **Dr. P.Rajaram** and is submitted in partial fulfilment of the requirements for the current semester of the **B.Tech., (AI&ML)** program at Saveetha Institute of Medical and Technical Sciences, Chennai.

SIGNATURE

Dr. John Justin

Professor and Program Director

Department of CSE

SIMATS Engineering

Thandalam

Chennai

SIGNATURE

Dr. P. Rajaram

Professor

Department of CSE

SIMATS Engineering

Thandalam

Chennai

Submitted for the Project work Viva-Voce held on _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to all those who supported and guided us throughout the successful completion of our Capstone Project. We are deeply thankful to our respected Founder and Chancellor, Dr. N.M. Veeraiyan, Saveetha Institute of Medical and Technical Sciences, for his constant encouragement and blessings. We also express our sincere thanks to our Pro-Chancellor, Dr. Deepak Nallaswamy Veeraiyan, and our Vice-Chancellor, Dr. Ashwani Kumar, for their visionary leadership and moral support during the course of this project.

We are truly grateful to our Director, Dr. Ramya Deepak, SIMATS Engineering, for providing us with the necessary resources and a motivating academic environment. Our special thanks to our Principal, Dr. B. Ramesh for granting us access to the institute's facilities and encouraging us throughout the process. We sincerely thank our Head of the Department, Dr. John Justin for his continuous support, valuable guidance, and constant motivation.

We are especially indebted to our guide, Dr .P.Rajaram for the creative suggestions, consistent feedback, and unwavering support during each stage of the project. We also express our gratitude to the Project Coordinators, Review Panel Members (Internal and External), and the entire faculty team for their constructive feedback and valuable inputs that helped improve the quality of our work. Finally, we thank all faculty members, lab technicians, our parents, and friends for their continuous encouragement and support.

Signature With Student Name

R. Pranathi(192525076)

M. Meghana(192525435)

M. Divyasree(192524098)

ABSTRACT

Cloud computing provides flexible and scalable computing resources, but manually managing virtual infrastructure can lead to resource wastage, performance issues, and increased operational effort. This project, titled “Cloud Based IaaS Virtual Infrastructure Provisioning and Auto Scaling System,” focuses on developing a cloud-based Infrastructure as a Service (IaaS) model that automatically provisions and manages virtual computing resources according to workload requirements.

The proposed system monitors resource parameters such as CPU utilization, memory usage, and workload demand to determine when additional or fewer resources are required. When the workload increases beyond a predefined threshold, the system performs scale-out operations by provisioning additional virtual resources.

When the workload decreases, scale-in operations reduce unnecessary resources. This approach minimizes manual intervention while improving resource utilization, scalability, availability, and cost efficiency. The project also examines the provisioning process, monitoring mechanism, auto-scaling logic, and performance under different workload conditions.

The proposed model demonstrates how automated infrastructure management can provide flexible cloud resources while responding dynamically to changing application demands. Overall, the project presents an efficient approach to managing virtual infrastructure using IaaS provisioning and auto-scaling concepts.

TABLE OF CONTENTS

S. No.	Title	Page No
CHAPTER 1		
1.1	Background Information	7
1.2	Project Objectives	7
1.3	Significance	8
1.4	Scope of Study	8
1.5	Methodology Overview	9
CHAPTER 2		
2.1	Description of the Problem	10
2.2	Evidence of the Problem	10
2.3	Stakeholders	11
2.4	Supporting Data Research	11
CHAPTER 3		
3.1	Development and Design Process	12
3.2	Tools and Technologies Used	12
3.3	Solution Overview	13
3.4	Engineering Standards Applied	13
3.5	Solution Justification	13
CHAPTER 4		
4.1	Evaluation of Results	14
4.2	Challenges Encountered	14
4.3	Possible Improvements	14
4.4	Recommendations	14
CHAPTER 5		
5.1	Key Learning Outcomes	15

5.2	Challenges Encountered and Overcome	15
5.3	Application of Engineering Standards	15
5.4	Insights into the Industry	15
5.5	Conclusion of Personal Development	16
6	Conclusion	16
7	References	17

LIST OF FIGURES

Figure No	Figure Title / Description
Figure 1	OpenStack Networking Verification (Horizon – Networks)
Figure 2	OpenStack Networking Verification (Horizon – Routers)
Figure 3	OpenStack Networking Verification (CLI – network/subnet/router list)
Figure 4	OpenStack Horizon Dashboard – Instances Panel
Figure 5	OpenStack Service Verification (openstack service list)
Figure 6	Nova Compute Sub-Service Verification (openstack compute service list)
Figure 7	VM Provisioning Verification (openstack server list --all-projects)
Figure 8	Hypervisor Verification (openstack hypervisor list)
Figure 9	Final VM Status (Horizon – Instances Panel, Post-Recovery)

LIST OF TABLES

Table No.	Table Title / Description
Table 1	Project Objectives
Table 2	OpenStack Services and Their Functions

Table 3	Hardware Requirements
Table 4	Software Requirements
Table 5	Functional Requirements
Table 6	Non-Functional Requirements
Table 7	Project Modules
Table 8	Implementation Verification
Table 9	Challenges and Solutions
Table 10	Possible Future Enhancements

LIST OF ABBREVIATIONS

Abbreviation	Expansion
IaaS	Infrastructure as a Service
PaaS	Platform as a Service
SaaS	Software as a Service
VM	Virtual Machine
API	Application Programming Interface
CLI	Command Line Interface
WSL	Windows Subsystem for Linux
QEMU	Quick Emulator
OOM	Out Of Memory
RAM	Random Access Memory
CPU	Central Processing Unit

CHAPTER 1 – INTRODUCTION

1. Background Information

Modern software systems increasingly depend on computing infrastructure that can be provisioned rapidly, scaled according to demand, and managed with minimal manual intervention. Traditional on-premises infrastructure requires organisations to purchase, install, and maintain physical servers, which is both capital-intensive and inflexible when workloads fluctuate. Cloud computing addresses this limitation by abstracting physical hardware into virtualised, on-demand resources that can be provisioned, modified, and released programmatically. This project explores the practical implementation of an Infrastructure as a Service (IaaS) environment using OpenStack, an open-source cloud computing platform widely adopted in both industry and research settings, and evaluates its suitability for automated virtual infrastructure provisioning and, conceptually, auto-scaling.

2. Introduction to Cloud Computing

Cloud computing refers to the on-demand delivery of computing resources, including servers, storage, networking, and software, over the internet on a pay-as-you-go or self-service basis. It is commonly categorised into three service models: Infrastructure as a Service (IaaS), which provides virtualised computing resources; Platform as a Service (PaaS), which provides a managed application development and deployment environment; and Software as a Service (SaaS), which delivers fully managed applications to end users. Cloud deployments may further be classified as public, private, or hybrid, depending on ownership and accessibility of the underlying infrastructure. This project focuses specifically on the IaaS model, implemented in a private, self-hosted manner using OpenStack.

3. Introduction to IaaS

Infrastructure as a Service (IaaS) is a cloud service model in which virtualised computing resources, such as virtual machines, storage volumes, and virtual networks, are provisioned and managed over the internet or a private network. Unlike PaaS or SaaS, IaaS gives the user direct control over the operating system and runtime environment of the provisioned virtual machines, while the underlying physical hardware, hypervisor, and virtualisation layer are managed by the

cloud platform. IaaS platforms typically expose this functionality through a combination of a command-line interface (CLI), a REST API, and a web-based dashboard, all of which were exercised in this project through OpenStack's Nova, Keystone, and Horizon services respectively.

Project Motivation

The motivation for this project arises from the practical need to understand how modern cloud platforms provision and manage virtual infrastructure internally, rather than relying solely on abstracted public-cloud interfaces. By deploying OpenStack in a local, resource-constrained environment, the project provides direct, hands-on exposure to the identity management, compute scheduling, networking, storage, and resource-tracking mechanisms that underlie commercial IaaS offerings. Additionally, the project is motivated by the broader industry trend towards elastic infrastructure, where auto-scaling and load balancing reduce both operational cost and manual administrative effort, and it seeks to understand the architectural components required to achieve this, even where full automation was not implemented within the scope of the present work.

4. Problem Statement

Manually provisioning, monitoring, and scaling virtual infrastructure is time-consuming, error-prone, and does not respond efficiently to changing workload conditions. Organisations that rely on manual infrastructure management often over-provision resources to accommodate peak demand, resulting in wasted capacity during periods of low utilisation, or under-provision resources, resulting in degraded performance during demand spikes. There is a need for a cloud-based IaaS environment that supports on-demand virtual machine provisioning, and that can, in principle, be extended with automated monitoring and scaling logic to adjust resource allocation dynamically in response to workload.

5. Project Objectives

The primary objectives of this project are outlined below and summarised in Table 1.

Table 1 – Project Objectives

S.No.	Objective
1	Deploy a functional OpenStack IaaS environment using DevStack on an Ubuntu/WSL platform.

S.No.	Objective
2	Configure and verify core OpenStack services: Keystone, Nova, Neutron, Glance, Cinder, Placement, and Horizon.
3	Provision and verify a virtual machine using a Cinder-backed boot volume and the QEMU hypervisor.
4	Configure and verify the Horizon web dashboard through Apache.
5	Design a conceptual auto-scaling and load-balancing mechanism based on workload monitoring.
6	Design a conceptual monitoring and billing dashboard for infrastructure usage.
7	Diagnose and resolve real-world infrastructure issues encountered during deployment.

7. Significance of the Project

This project is significant because it demonstrates, through direct implementation, how the constituent services of a modern IaaS platform interoperate to deliver a functioning virtual machine to an end user. It also demonstrates the practical challenges of operating cloud infrastructure under constrained hardware resources, an experience that mirrors real-world capacity-planning and reliability concerns faced by cloud administrators. The distinction maintained throughout the report between implemented functionality and proposed/conceptual functionality is intended to reflect the rigour expected of an engineering capstone evaluation.

8. Scope of the Project

The scope of this project covers the deployment and verification of core OpenStack IaaS services and the provisioning of a single virtual machine instance within a DevStack-based development environment. The scope also includes the conceptual design, but not the full automated

implementation, of an auto-scaling and load-balancing engine, and a monitoring and billing dashboard. The project does not extend to multi-node OpenStack deployment, production-grade high-availability configuration, or integration with public cloud providers, all of which are identified as directions for future work.

9. Methodology Overview

The project methodology followed a structured approach consisting of environment setup, service deployment, functional verification, and troubleshooting. OpenStack DevStack was deployed on an Ubuntu environment running under the Windows Subsystem for Linux (WSL), using QEMU as the hypervisor. Core services were verified using the OpenStack command-line client, following which a virtual machine was provisioned from a Cinder-backed volume and its status was verified through both the CLI and, where accessible, the Horizon dashboard. During this process, a significant memory-related fault was encountered and resolved, and the resulting troubleshooting methodology is documented as part of the results of this project.

10. Organization of the Report

This report is organised into six chapters. Chapter 1 introduces the project background, motivation, objectives, and scope. Chapter 2 identifies and analyses the problem being addressed, along with the existing and proposed systems. Chapter 3 describes the solution design and implementation, including system architecture and the three project modules. Chapter 4 presents the implementation results, verification evidence, and the troubleshooting case study. Chapter 5 reflects on the learning outcomes and personal development achieved through the project. Chapter 6 concludes the report and outlines directions for future work.

CHAPTER 2 – PROBLEM IDENTIFICATION AND ANALYSIS

1. Description of the Problem

Traditional infrastructure provisioning is a manual, administrator-driven process that involves procuring physical or virtual servers, configuring networking and storage, and monitoring resource usage without automated feedback. This approach does not scale efficiently with fluctuating application demand and often results in either resource wastage or performance degradation. The problem addressed by this project is the absence of an accessible, self-hosted IaaS environment that demonstrates automated virtual machine provisioning and that is architecturally prepared to support automated scaling and monitoring.

2. Existing System

In many academic and small-scale organisational settings, virtual machines are provisioned manually using desktop virtualisation tools such as VirtualBox or VMware Workstation, without a centralised identity, networking, or resource-management layer. Public cloud providers such as AWS, Microsoft Azure, and Google Cloud Platform offer mature IaaS capabilities, including automated scaling, but operate as proprietary, metered services that do not expose their internal architecture and are not freely accessible for detailed academic study of the underlying provisioning mechanisms.

3. Limitations of Existing System

- Manual VM provisioning is slow and prone to configuration inconsistency.
- Desktop virtualisation tools lack centralised identity management, networking abstraction, and resource tracking.
- Public cloud IaaS platforms are cost-incurring and do not expose internal service architecture for study purposes.
- Absence of automated monitoring makes it difficult to detect resource exhaustion before it causes service failure.
- No standard mechanism exists in ad hoc setups for scaling resources in response to workload changes.

4. Proposed System

The proposed system uses OpenStack, deployed via DevStack, to provide a self-hosted IaaS environment with centralised identity management (Keystone), compute provisioning (Nova), networking (Neutron), image management (Glance), block storage (Cinder), resource tracking (Placement), and a web-based management dashboard (Horizon). On top of this implemented provisioning layer, the system proposes a conceptual auto-scaling and load-balancing engine that monitors resource utilisation and adjusts the number of active virtual machines accordingly, together with a monitoring and billing dashboard that tracks infrastructure usage.

5. Advantages of Proposed System

- Provides a unified, API-driven interface for provisioning and managing virtual infrastructure.
- Uses open-source components, avoiding proprietary cost and vendor lock-in.
- Exposes the internal architecture of an IaaS platform, supporting deeper technical understanding.
- Establishes a foundation that can be extended with OpenStack Heat, Telemetry/Aodh, and Octavia for full automation.
- Demonstrates real-world fault diagnosis and recovery within a constrained-resource deployment.

6. Stakeholders

Stakeholder	Need / Concern
Application developers	Require rapid, on-demand access to virtual machines for development and testing.
Infrastructure administrators	Require centralised visibility and control over compute, storage, and network resources.

Stakeholder	Need / Concern
-------------	----------------

Academic evaluators	Require clear evidence distinguishing implemented functionality from proposed design.
End users of hosted applications	Require consistent application performance despite workload fluctuation.
Future project contributors	Require a documented, extensible architecture for adding automation.

7. Functional Requirements Table

5 – Functional Requirements

ID	Requirement
FR1	The system shall authenticate users and services through Keystone.
FR2	The system shall allow provisioning of a virtual machine with a specified flavor and boot source.
FR3	The system shall allocate block storage to a virtual machine via Cinder.
FR4	The system shall configure a virtual network for provisioned instances via Neutron.
FR5	The system shall provide a web dashboard for infrastructure management via Horizon.
FR6	The system shall report the status of compute services and hypervisors via the CLI.
FR7 (Proposed)	The system shall, in a future implementation, monitor workload and trigger scale-out/scale-in actions.

8. Non-Functional Requirements Table

6 – Non-Functional Requirements

ID	Requirement
NFR1	The system should provision a virtual machine within a reasonable time under adequate resource availability.
NFR2	The system should remain responsive to CLI and dashboard requests under normal load.
NFR3	The system should be resilient to transient resource pressure, or clearly report failure conditions.
NFR4	The system architecture should be modular, allowing individual services to be scaled or replaced.
NFR5	The system should log sufficient diagnostic information to support troubleshooting.

9. Supporting Technical Research

OpenStack is documented extensively by the OpenStack Foundation as a modular, service-oriented cloud operating system, in which independent services communicate over REST APIs and a shared message queue. Industry literature on cloud elasticity commonly identifies threshold-based monitoring, coupled with automated provisioning APIs, as the standard mechanism for achieving auto-scaling, an approach reflected in the conceptual design adopted in this project for OpenStack Telemetry/Aodh-based monitoring combined with Heat-based orchestration. Similarly, published OpenStack documentation identifies Octavia as the standard Load-Balancing-as-a-Service component for distributing traffic across scaled instances.

CHAPTER 3 – SOLUTION DESIGN AND IMPLEMENTATION

1. Overall System Architecture

The system architecture follows the standard OpenStack service-oriented model, in which a user interacts with the platform through either the Horizon web dashboard or the OpenStack command-line client. Requests are authenticated by Keystone before being routed to the appropriate service: Nova for compute provisioning, Neutron for networking, Glance for image retrieval, Cinder for block storage, and Placement for resource allocation tracking. The Nova compute service ultimately delegates virtual machine execution to the QEMU hypervisor.

2. OpenStack Architecture

OpenStack is composed of loosely coupled services that communicate through REST APIs and an internal message bus, backed by a shared database for persistent state. Each service exposes its own API endpoint registered with Keystone's service catalogue, allowing the CLI, SDKs, and Horizon dashboard to discover and interact with the appropriate service without hard-coded configuration. This modular design allows individual services to be scaled, replaced, or extended independently, which is a key architectural property that supports the future integration of orchestration and monitoring services such as Heat and Telemetry/Aodh.

3. System Workflow

The following sequence describes the logical workflow by which a virtual machine request travels through the OpenStack architecture, from user request to running instance:

User → Horizon / CLI → Keystone Authentication → Nova Compute → Placement Resource Allocation → Glance Image / Cinder Storage → Neutron Networking → QEMU Hypervisor → Virtual Machine

The user initiates a request through either the Horizon dashboard or the OpenStack CLI. Keystone authenticates the request and issues a token used to authorise subsequent API calls. Nova receives the compute request and queries Placement to determine which host has sufficient resources (CPU, memory, disk) to satisfy the requested flavor. Nova then coordinates with Glance to retrieve the boot image, or with Cinder where the instance boots from a persistent volume, and with Neutron

to attach a virtual network interface. Finally, Nova instructs the QEMU hypervisor to instantiate and start the virtual machine.

4. Module 1 – VM Provisioning & Resource Allocation

Module 1 is the core implemented module of this project. It is responsible for authenticating administrative access, provisioning a virtual machine from a Cinder-backed boot volume, and allocating compute resources through Nova and Placement. This module was implemented and verified: a virtual machine, named vm1, was successfully created using the m1.tiny flavor, configured to boot from a Cinder volume, and executed under the QEMU hypervisor. The instance was verified to reach the ACTIVE state, confirming successful resource allocation and instantiation.

5. Module 2 – Auto-Scaling & Load Balancing Engine

Module 2 is presented in this project as a conceptual and design-stage module; it was not implemented or experimentally tested within the current scope of work. The intended design monitors workload indicators such as CPU utilisation, memory usage, and request volume, and compares these against predefined thresholds. When the workload exceeds an upper threshold, the engine would provision an additional virtual machine and distribute incoming workload across the available instances. When the workload falls below a lower threshold, the engine would remove or reduce unnecessary virtual machine instances to conserve resources. Section 3.15 provides a detailed description of this proposed mechanism, and Section 4 clearly distinguishes it from the results that were actually obtained.

6. Module 3 – Monitoring & Billing Dashboard

Module 3 covers infrastructure monitoring and cost/usage tracking. The monitoring aspect of this module was partially verified: service and hypervisor status, and VM state, were confirmed through OpenStack CLI diagnostics such as `openstack compute service list` and `openstack hypervisor list`. The billing/cost-tracking aspect of this module, which would track VM runtime, resource allocation, storage usage, network usage, and instance count for cost-estimation purposes, remains a proposed design component and was not implemented or populated with real billing data in this project.

7. OpenStack Services Used

Table 2 – OpenStack Services and Their Functions

Service	Function
Keystone	Identity and authentication service; issues tokens and maintains the service catalogue.
Nova	Compute service; manages virtual machine lifecycle and scheduling.
Neutron	Networking service; manages virtual networks, subnets, and ports.
Glance	Image service; manages virtual machine disk images.
Cinder	Block storage service; manages persistent storage volumes attached to instances.
Placement	Resource tracking service; tracks available and consumed compute resources.
Horizon	Web-based dashboard for managing OpenStack resources.

8. Hardware and Software Requirements

Table 3 – Hardware Requirements

Component	Specification
RAM	Approximately 7.6 GiB available (observed during deployment)
Processor	Multi-core x86_64 processor with virtualization support
Storage	Sufficient free disk space for DevStack, images, and volumes
Swap Space	Increased swap capacity (required after troubleshooting, see Section 4.9)

Table 4 – Software Requirements

Component	Specification
Host OS	Windows with Windows Subsystem for Linux (WSL)
Guest OS	Ubuntu (Linux distribution running under WSL)
Cloud Platform	OpenStack, deployed via DevStack
Hypervisor	QEMU
Web Server	Apache (hosting the Horizon dashboard)
Database	MySQL (OpenStack service backend)
CLI Tooling	OpenStack unified command-line client (python-openstackclient)

9. Development Environment

The development environment consisted of an Ubuntu distribution running under the Windows Subsystem for Linux (WSL) on a Windows host machine. OpenStack was deployed using DevStack, a set of scripts that installs and configures OpenStack services from source for development and testing purposes. This environment was chosen because it allows a complete, multi-service OpenStack deployment to be run on a single host without dedicated server hardware, at the cost of reduced available memory and compute headroom compared to a production multi-node deployment.

10. Implementation Procedure

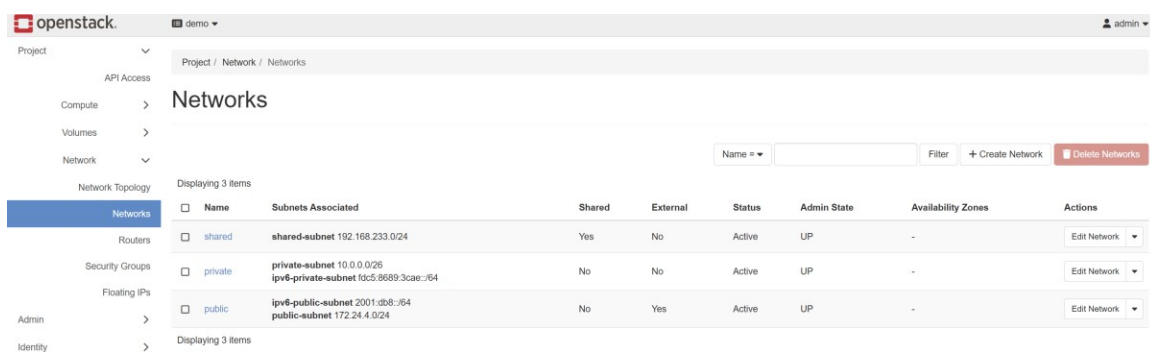
The implementation proceeded in the following stages: installation and configuration of DevStack; verification of core service availability using the OpenStack CLI; provisioning of a virtual machine from a Cinder-backed volume; configuration and verification of the Horizon dashboard through Apache; and diagnosis and resolution of a memory-exhaustion fault encountered during testing. Each of these stages is described in further detail in the following subsections and in Chapter 4.

11. VM Provisioning Process

The virtual machine, named `vm1`, was provisioned using the `m1.tiny` flavor, a minimal resource profile suited to constrained development environments. The instance was configured to boot from a Cinder-backed volume rather than an ephemeral disk, meaning that persistent block storage managed by Cinder was used as the instance's boot device. This choice demonstrates the integration between Nova (compute), Cinder (storage), and Placement (resource allocation) in the provisioning workflow, and reflects a design pattern commonly used in production IaaS deployments where boot-from-volume improves data persistence and enables features such as volume snapshotting.

12. Networking Configuration

Networking for the provisioned instance was managed by Neutron, which is responsible for allocating virtual network ports, managing IP addressing, and connecting instances to the appropriate virtual network. The DevStack default networking configuration was verified directly: three networks (shared, private, and public) were confirmed as `ACTIVE` with their associated subnets, and a router (router1) connecting the private network to the public external network was confirmed as `ACTIVE`. This was verified both through the OpenStack CLI (`openstack network list` / `openstack subnet list` / `openstack router list`) and through the Horizon dashboard's Network panel. The successful provisioning and `ACTIVE` status of `vm1`, with an assigned IPv4 and IPv6 address on the private network, confirms that the instance was assigned functioning network connectivity.



The screenshot shows the OpenStack Horizon dashboard with the 'Networks' panel selected. The dashboard header includes the OpenStack logo, a 'demo' dropdown, and a user profile 'admin'. The left sidebar shows navigation options: Project, API Access, Compute, Volumes, Network, Network Topology, Routers, Security Groups, Floating IPs, Admin, and Identity. The main content area displays a table of networks with columns for Name, Subnets Associated, Shared, External, Status, Admin State, Availability Zones, and Actions. Three networks are listed: 'shared', 'private', and 'public', all with a status of 'Active'.

Name	Subnets Associated	Shared	External	Status	Admin State	Availability Zones	Actions
shared	shared-subnet 192.168.233.0/24	Yes	No	Active	UP	-	Edit Network
private	private-subnet 10.0.0.0/26 ipv6-private-subnet fdcd:8689:3cae::/64	No	No	Active	UP	-	Edit Network
public	ipv6-public-subnet 2001:db8::/64 public-subnet 172.24.4.0/24	No	Yes	Active	UP	-	Edit Network

Figure 1 – OpenStack Networking Verification (Horizon – Networks)

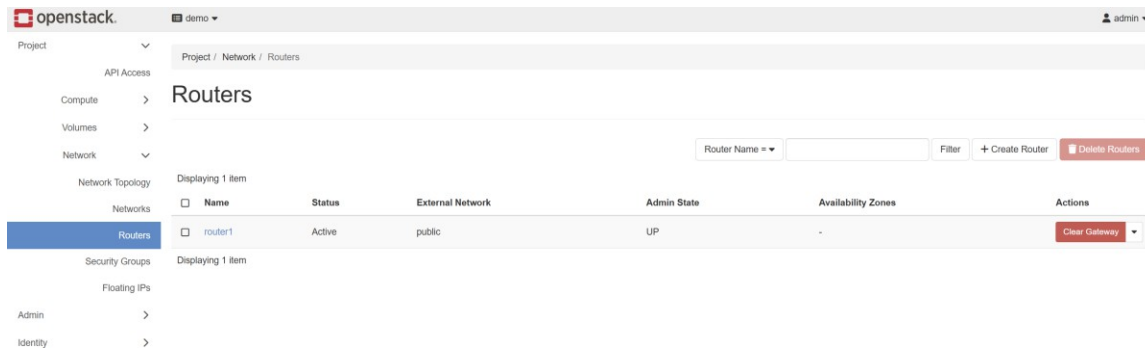


Figure 2 – OpenStack Networking Verification (Horizon – Routers)

```
divya_sree_m@LAPTOP-EDH45F4T:~/devstack/devstack$ openstack network list
openstack subnet list
openstack router list
```

ID	Name	Subnets
853ec43d-13ae-44a0-983e-92958b024417	shared	fa3dd3bf-d218-4fb7-95ac-fb3235d74c3f
8ee45ca0-4bbb-4b17-b3fa-cf4cfeaa0cf1	public	2e2de05f-d2cb-4fe7-b11e-d901b4f9ed85, 660731f8-7c2d-4211-ba27-84e22e247f66
d603f001-84c7-414e-979d-ecb1279479d1	private	4ef8330d-0686-4fee-9626-21cd4fedfaef, b7803708-d316-45f7-a396-5a750fb9c821

Figure 3 – OpenStack Networking Verification (CLI – network/subnet/router list)

13. Storage Configuration

Persistent block storage was provided through Cinder, which was configured to supply the boot volume for vm1. This configuration was verified indirectly through the successful provisioning and ACTIVE state of the instance, and through the presence of the cinder service in the output of the openstack service list command.

14. Dashboard Configuration

The Horizon web dashboard was configured to run through the Apache web server. The Apache service was verified as active using `sudo systemctl status apache2 --no-pager`, and the dashboard endpoint was tested using `curl -I http://127.0.0.1/dashboard/`, which returned an HTTP 302 redirect

to the Horizon login page. This response confirms that the dashboard endpoint was correctly deployed and responding to local HTTP requests, even though it redirects unauthenticated requests to the login page as expected. Beyond the endpoint-level check, the dashboard was also accessed directly and used to browse the Compute and Network panels, confirming that authenticated access, instance listing, and network/router listing all functioned correctly through the web interface.

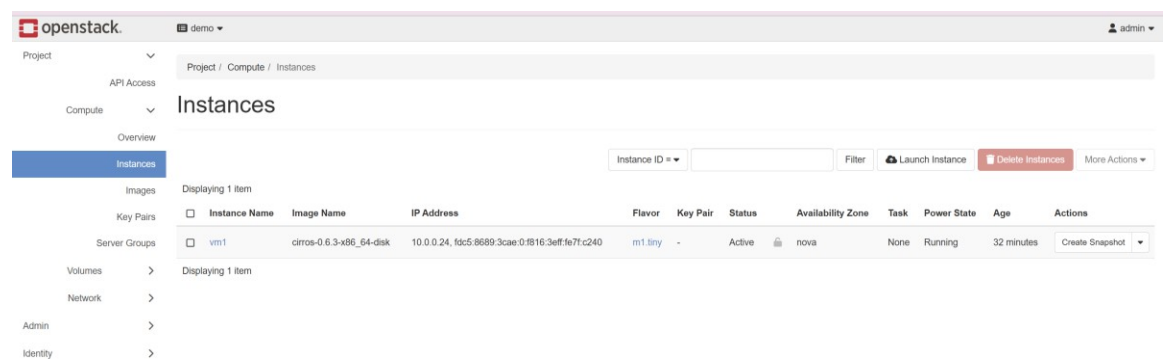


Figure 4 – OpenStack Horizon Dashboard – Instances Panel

15. Auto-Scaling Design (Proposed)

The proposed auto-scaling mechanism is described conceptually as follows. Under conditions of high workload, the monitoring component would continuously observe CPU utilisation, memory usage, and overall workload demand; when a predefined upper threshold is exceeded, the system would provision an additional virtual machine and distribute incoming workload across the expanded pool of instances, while monitoring continues. Under conditions of low workload, resource utilisation would be observed to decrease; if utilisation remains below a lower threshold for a sustained period, the system would remove or reduce unnecessary virtual machine instances to conserve resources. This logic was not implemented or tested in the current project; it is

presented purely as a design specification for future work. OpenStack Heat could provide the orchestration templates needed to automate instance creation and deletion, OpenStack Telemetry/Aodh could provide the metric collection and alarm-triggering mechanism, and OpenStack Octavia could provide Load-Balancing-as-a-Service to distribute traffic across the resulting instance pool.

16. Monitoring Design

The monitoring module is designed to track VM status, CPU utilisation, memory utilisation, resource allocation, network status, service availability, and overall infrastructure health. In the current implementation, service and hypervisor availability, and VM instance status, were verified through OpenStack CLI diagnostic commands, as documented in Section 3.10 and Chapter 4. Continuous, automated collection of CPU and memory metrics through OpenStack Telemetry/Aodh was not implemented and is identified as a proposed extension.

17. Engineering Standards / Best Practices

Standard / Practice	Purpose
OpenStack API versioning conventions	Ensures compatibility between CLI/SDK clients and service endpoints
Principle of least privilege (Keystone roles)	Restricts administrative operations to authorised users
Infrastructure as Code principles	Supports reproducible service configuration and future orchestration via Heat
Resource capacity planning	Guides appropriate sizing of RAM and swap for service stability

18. Security Considerations

Security in the implemented environment relies primarily on Keystone's identity and token-based authentication, which ensures that only authenticated users and services can invoke privileged OpenStack API operations. Administrative credentials were sourced through the OpenStack RC

file (openrc) rather than hard-coded into scripts. In a production deployment, additional considerations such as network segmentation via Neutron security groups, encrypted communication between services, and role-based access control refinement would be necessary; these were outside the primary scope of the present development-environment deployment.

19. Solution Justification

- OpenStack is a widely adopted, open-source IaaS platform, providing transparency into service architecture unavailable in proprietary public clouds.
- DevStack allows a full multi-service deployment on a single development host, suitable for academic study.
- Boot-from-volume configuration via Cinder demonstrates integration between compute, storage, and resource-allocation services.
- The modular OpenStack architecture provides a clear extension path towards Heat, Telemetry/Aodh, and Octavia for future automation.
- CLI-based verification provides auditable, reproducible evidence of service and instance status.

CHAPTER 4 – RESULTS AND RECOMMENDATIONS

1. Implementation Results

This section presents the results actually obtained during implementation. All results reported here were directly observed through OpenStack CLI output or dashboard behaviour; no experimental values relating to auto-scaling, load balancing, or billing have been included, as these components were not implemented in the current scope of the project.

2. OpenStack Service Verification

Core OpenStack service availability was verified using the command `openstack service list`, executed after sourcing the administrative credentials with `source ~/devstack/devstack/openrc` `admin admin`. The output confirmed the presence and registration of the cinder, neutron, glance, nova, placement, keystone, and nova_legacy services within the service catalogue, indicating that the core IaaS services were successfully installed and registered.

```
divya_sree_m@LAPTOP-EDH45F4T:~/devstack/devstack$ openstack service list
```

ID	Name	Type
95a24b36e8b94d18af16cac186182ddb	cinder	block-storage
9e7533cc7edb40cf8cc36314d3bdf797	neutron	network
a8ba67056a0e46d9822cf6ee2f6615f2	glance	image
a9488ef42ff2487eb301bcde44629986	nova	compute
aa3815cbb9c44606ad018e071ca25ae3	placement	placement
ace30e626beb416793884b91e2e4e6d4	keystone	identity
fd0acb3f7c604443a2d077828b105d22	nova_legacy	compute_legacy

Figure 5 – OpenStack Service Verification (openstack service list)

In addition to the overall service catalogue, the Nova compute sub-services were individually verified using `openstack compute service list`, which confirmed that the nova-scheduler, nova-conductor, and nova-compute processes were all reported as enabled and up on the deployment host.

```
divya_sree_m@LAPTOP-EDH45F4T:~/devstack/devstack$ openstack compute service list
```

ID	Binary	Host	Zone	Status	State	Updated At
6130d750-78d2-41b1-b064-22a2ebbc68eb	nova-scheduler	LAPTOP-EDH45F4T	internal	enabled	up	2026-08-20T05:28:47.000000
32a23a33-9871-4a00-ae94-2bc8f9caad0b	nova-conductor	LAPTOP-EDH45F4T	internal	enabled	up	2026-08-20T05:28:47.000000
74377916-8a31-4d0b-9da2-6529c5ae6cbf	nova-conductor	LAPTOP-EDH45F4T	internal	enabled	up	2026-08-20T05:29:11.000000
cb3e7d3b-0b1b-4ee4-98a6-30d51a6321de	nova-compute	LAPTOP-EDH45F4T	nova	enabled	up	2026-08-20T05:28:48.000000

Figure 6 – Nova Compute Sub-Service Verification (openstack compute service list)

3. VM Provisioning Results

The virtual machine vm1 was provisioned using the m1.tiny flavor with a Cinder-backed boot volume. The instance was verified using the command `openstack server list --all-projects`, which confirmed that vm1 was present in the OpenStack environment and reported in the ACTIVE state, indicating successful provisioning and startup. The instance was independently cross-verified through the Horizon dashboard's Instances panel, which likewise reported vm1 as Active with a Running power state on the private network.

```
divya_sree_m@LAPTOP-EDH45F4T:~/devstack/devstack$ openstack server list --all-projects
```

ID	Name	Status	Networks	Image	Flavor	Project ID
c448dca4-7c8a-4878-8ee8-dfb11ded7838	vm1	ACTIVE	private=10.0.0.24, fdc5:8689:3cae:0:f816:3eff:fe7f:c240	N/A (booted from volume)	m1.tiny	3a38c3c2ff4b4d53a80a37653a047f6a

Figure 7 – VM Provisioning Verification (openstack server list --all-projects)

4. Hypervisor Verification

The QEMU hypervisor was verified using the command `openstack hypervisor list`, which reported the hypervisor as UP. This confirms that the Nova compute service correctly detected and registered the underlying virtualization layer responsible for executing provisioned instances.

```
divya_sree_m@LAPTOP-EDH45F4T:~/devstack/devstack$ openstack hypervisor list
```

ID	Hypervisor Hostname	Hypervisor Type	Host IP	State
a78710d8-fb4d-43e7-b8a2-d6d1a8454a22	LAPTOP-EDH45F4T.localdomain	QEMU	172.21.30.246	up

Figure 8 – Hypervisor Verification (openstack hypervisor list)

5. Dashboard Verification

The Horizon dashboard, served through Apache, was verified using `curl -I http://127.0.0.1/dashboard/`, which returned an HTTP 302 redirect response directing to the Horizon login page. This confirms that the dashboard endpoint was correctly deployed and reachable on the local host. The Apache service itself was confirmed active using `sudo systemctl status apache2 --no-pager`.

6. Network and Storage Verification

Network and storage integration were verified indirectly through the successful provisioning of vm1 from a Cinder-backed volume with functioning network connectivity, as well as through the presence of the neutron and cinder services in the service list output described in Section 4.2. Detailed per-port or per-volume diagnostic output was not separately captured as part of this project's verification scope.

7. Performance Observations

Under normal operating conditions, following resolution of the memory-exhaustion issue described in Section 4.9, OpenStack CLI commands returned responsive output and the provisioned instance remained in the ACTIVE state. No formal performance benchmarking, such as measured provisioning latency or throughput under concurrent load, was conducted, and no such figures are reported here, consistent with the project's accuracy requirements.

8. Challenges Encountered

Table 9 – Challenges and Solutions

Challenge	Solution
Running a full OpenStack deployment within a resource-constrained WSL environment	Careful sequencing of service startup and increased swap allocation
openstack server list hanging and Horizon becoming inaccessible	Diagnosed as memory exhaustion; resolved by increasing swap and restarting affected services
MySQL terminated by the Linux OOM killer	Increased swap capacity to relieve memory pressure and allowed MySQL to restart cleanly
Verifying dashboard reachability without a full browser-based login flow	Used curl -I to confirm HTTP-level endpoint availability

9. Root Cause Analysis of System Failure

During implementation, the OpenStack environment experienced severe performance degradation, characterised by the `openstack server list` command hanging, the Horizon dashboard becoming inaccessible, HTTP requests timing out, and OpenStack API commands becoming slow to respond. Investigation using the `free -h` command revealed that system memory, approximately 7.6 GiB of RAM, was almost fully consumed, and that swap space was completely exhausted. System logs further revealed that the MySQL service, which backs the databases used by Keystone, Nova, Neutron, and other OpenStack services, had been terminated by the Linux Out-Of-Memory (OOM) killer, with the system reporting: “mysql.service: A process of this unit has been killed by the OOM killer.” Because nearly all OpenStack services depend on database connectivity, the loss of MySQL caused dependent API calls to fail or hang, producing the observed symptoms across the CLI and dashboard.

10. Troubleshooting and Recovery

The investigation proceeded systematically: the Apache web server status was checked, followed by the status of the core OpenStack services, followed by the database service, and finally by overall system memory usage using `free -h`, which revealed the underlying RAM and swap exhaustion. Once memory exhaustion was identified as the root cause, swap capacity was increased and the affected services were restarted. Following this remediation, OpenStack commands became responsive again. The command `openstack compute service list` subsequently showed the Nova scheduler, Nova conductor, and Nova compute services as `ENABLED` and `UP`, the hypervisor was again shown as `UP`, and `openstack server list --all-projects` confirmed that `vm1` remained present and correctly registered in the environment. The Horizon dashboard was likewise re-verified after recovery, continuing to report `vm1` as `Active` well beyond the point of the initial fault, confirming that the instance itself survived the underlying infrastructure disruption.

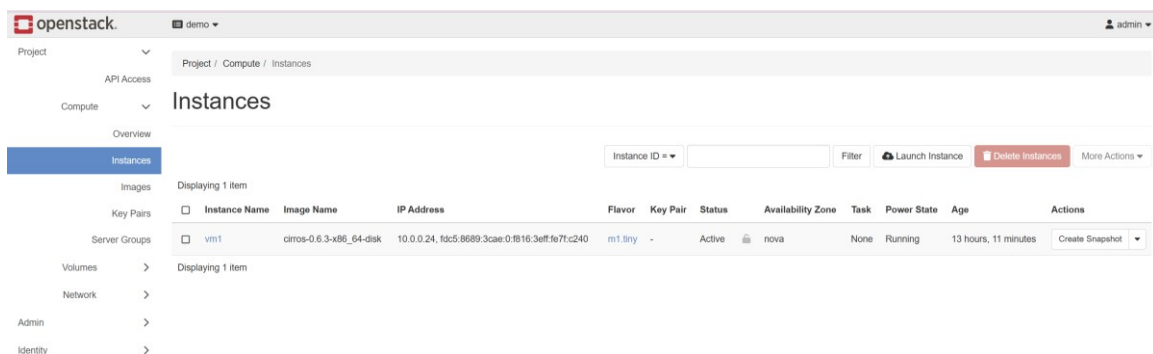


Figure 9 – Final VM Status (Horizon – Instances Panel, Post-Recovery)

This troubleshooting experience is considered a significant outcome of the project, as it demonstrates practical, real-world cloud infrastructure debugging: the ability to correlate application-level symptoms (hanging CLI commands, an inaccessible dashboard) with an underlying infrastructure-level cause (memory exhaustion and an OOM-killed database process), and to apply an appropriate remediation. It further illustrates why proper resource allocation and capacity planning, particularly of RAM and swap, are critical considerations when operating cloud infrastructure in resource-constrained environments such as a WSL-hosted DevStack deployment.

11. Possible Improvements

Table 10 – Possible Future Enhancements

Enhancement	Description
Implement OpenStack Telemetry/Aodh	Enable automated collection of CPU/memory metrics and threshold-based alarms
Implement OpenStack Heat	Enable automated, template-driven provisioning and de-provisioning of instances
Implement OpenStack Octavia	Enable Load-Balancing-as-a-Service across scaled instance pools
Deploy on a dedicated multi-node cluster	Reduce resource contention observed in the single-host WSL deployment
Increase host RAM and swap allocation	Prevent recurrence of OOM-related service failures
Implement a real usage-based billing module	Track VM runtime, storage, and network usage for cost estimation

12. Recommendations

- Allocate sufficient RAM and swap before deploying DevStack, based on observed memory pressure during this project.
- Introduce automated monitoring (Telemetry/Aodh) before attempting to implement auto-scaling logic, since scaling decisions depend on reliable metric collection. Adopt Heat templates for orchestration to make future scaling actions repeatable and auditable.
- Extend verification procedures to include Neutron port-level and Cinder volume-level diagnostics for more granular evidence.
- Consider a multi-node deployment for any future work targeting realistic performance benchmarking.

CHAPTER 5 – REFLECTION ON LEARNING AND PERSONAL DEVELOPMENT

1. Key Learning Outcomes

This project provided direct, practical exposure to the architecture and operation of a modern IaaS platform. Working through the deployment of OpenStack DevStack clarified how identity management, compute scheduling, networking, image handling, block storage, and resource tracking are implemented as independent, interoperating services rather than as a single monolithic system, and demonstrated how these services are orchestrated to provision a functioning virtual machine.

2. Technical Skills Acquired

- Practical use of the OpenStack command-line client for service and instance verification.
- Understanding of hypervisor-level virtualization concepts using QEMU.
- Configuration and diagnostic verification of a web dashboard (Horizon) served through Apache.
- Linux system administration skills, including memory diagnostics using `free -h` and service management using `systemctl`.
- Root-cause analysis of a multi-service failure caused by an operating-system-level resource constraint (OOM killer).

3. Challenges Encountered and Overcome

The most significant challenge encountered was the memory-exhaustion incident described in Chapter 4, which required correlating symptoms observed at the application layer (hanging commands, an inaccessible dashboard) with a root cause at the operating-system layer (RAM and swap exhaustion leading to an OOM-killed MySQL process). Overcoming this challenge required a systematic, layer-by-layer diagnostic approach rather than assuming the fault lay within OpenStack configuration itself.

4. Application of Engineering Practices

The project applied structured engineering practices including systematic requirements definition (Chapter 2), modular architectural design aligned with the underlying OpenStack service model (Chapter 3), and evidence-based verification of implemented functionality (Chapter 4). A deliberate effort was made throughout the report to separate implemented, verified functionality from proposed or conceptual design, reflecting sound engineering documentation practice.

5. Teamwork and Collaboration

The project was carried out collaboratively, with responsibilities distributed across environment setup, service verification, dashboard configuration, and troubleshooting, requiring coordination and shared understanding of the OpenStack architecture among team members. [INSERT SPECIFIC TEAM CONTRIBUTION DETAILS IF REQUIRED]

6. Problem-Solving Skills

The troubleshooting case study documented in Section 4.9 and 4.10 demonstrates applied problem-solving: forming a hypothesis about the failure's cause, gathering diagnostic evidence to test that hypothesis, and applying a targeted remediation rather than a broad, unverified fix. This experience is directly transferable to professional cloud-infrastructure operations roles.

7. Industry Insights

The project provided insight into why organisations increasingly adopt IaaS platforms with automated scaling and monitoring: manual infrastructure management does not scale efficiently, and resource-related failures, such as the OOM-killer incident observed in this project, can propagate across multiple dependent services if not proactively monitored. This reinforces the industry rationale for tools such as OpenStack Telemetry/Aodh, Heat, and Octavia, which this project has identified as the appropriate extension path rather than attempting to reproduce ad hoc.

8. Personal Development

Beyond the technical skills acquired, the project developed the discipline of accurately reporting what was implemented versus what was designed or proposed, an important professional habit when presenting engineering work to evaluators or stakeholders. It also built confidence in independently diagnosing and resolving infrastructure-level faults under time and resource constraints.

9. Future Learning

Future learning arising from this project includes deeper study of OpenStack Telemetry/Aodh for metrics-based alarms, OpenStack Heat for orchestration templates, and OpenStack Octavia for load balancing, all of which were identified but not implemented in the current work. Further learning is also planned around multi-node OpenStack deployment and production-grade high-availability configuration.

CHAPTER 6 – CONCLUSION

1. Project Summary

This project implemented and verified a functional OpenStack-based IaaS environment using DevStack on an Ubuntu/WSL platform, with QEMU as the underlying hypervisor. Core OpenStack services, Keystone, Nova, Neutron, Glance, Cinder, Placement, and Horizon, were deployed and verified through CLI diagnostics, and a virtual machine (vm1) was successfully provisioned from a Cinder-backed volume and confirmed to reach the ACTIVE state. In parallel, the project presented a conceptual design for an auto-scaling and load-balancing engine and a monitoring and billing dashboard, clearly distinguishing these design-stage proposals from the verified implementation results.

2. Achievement of Objectives

Of the objectives set out in Section 1.6, the objectives relating to OpenStack deployment, core service verification, virtual machine provisioning, and dashboard configuration were fully achieved and verified. The objectives relating to auto-scaling, load balancing, and a functioning billing module were achieved only at the design/conceptual level, as explicitly required by the project's accuracy constraints, and are appropriately reported as proposed future work rather than as completed implementation.

3. Overall Conclusion

The project demonstrates that a self-hosted OpenStack IaaS environment can be successfully deployed and operated within a constrained, single-host development environment, and that the constituent services of such a platform, identity, compute, networking, image, storage, and resource-tracking, can be verified through systematic CLI-based diagnostics. It further demonstrates that infrastructure reliability depends critically on adequate resource allocation, as evidenced by the memory-exhaustion incident that was diagnosed and resolved during the project. While full automated auto-scaling, load balancing, and billing were not implemented, the project establishes both the architectural foundation and a clear, technically justified roadmap for their future integration using OpenStack Heat, Telemetry/Aodh, and Octavia.

4. Future Scope

- Implement OpenStack Telemetry/Aodh for automated metric collection and threshold-based alarms.
- Implement OpenStack Heat to automate provisioning and de-provisioning of instances in response to alarms.
- Implement OpenStack Octavia to provide Load-Balancing-as-a-Service across auto-scaled instance pools.
- Deploy the environment on a multi-node cluster to evaluate performance under realistic, less resource-constrained conditions.
- Develop a real usage-based billing module that tracks VM runtime, storage consumption, and network usage.
- Conduct formal performance benchmarking of provisioning latency and scaling responsiveness once automation is implemented.

REFERENCES

1. OpenStack Foundation, “OpenStack Documentation,” OpenStack Docs. [Online]. Available: <https://docs.openstack.org/>
2. OpenStack Foundation, “Nova, the OpenStack Compute Service Documentation,” OpenStack Docs. [Online]. Available: <https://docs.openstack.org/nova/latest/>
3. OpenStack Foundation, “Neutron, the OpenStack Networking Service Documentation,” OpenStack Docs. [Online]. Available: <https://docs.openstack.org/neutron/latest/>
4. OpenStack Foundation, “Keystone, the OpenStack Identity Service Documentation,” OpenStack Docs. [Online]. Available: <https://docs.openstack.org/keystone/latest/>
5. OpenStack Foundation, “Cinder, the OpenStack Block Storage Service Documentation,” OpenStack Docs. [Online]. Available: <https://docs.openstack.org/cinder/latest/>
6. OpenStack Foundation, “Glance, the OpenStack Image Service Documentation,” OpenStack Docs. [Online]. Available: <https://docs.openstack.org/glance/latest/>
7. OpenStack Foundation, “Placement, the OpenStack Resource Tracking Service Documentation,” OpenStack Docs. [Online]. Available: <https://docs.openstack.org/placement/latest/>
8. OpenStack Foundation, “Horizon, the OpenStack Dashboard Documentation,” OpenStack Docs. [Online]. Available: <https://docs.openstack.org/horizon/latest/>
9. OpenStack Foundation, “Heat, the OpenStack Orchestration Service Documentation,” OpenStack Docs. [Online]. Available: <https://docs.openstack.org/heat/latest/>
10. OpenStack Foundation, “Octavia, the OpenStack Load Balancing Service Documentation,” OpenStack Docs. [Online]. Available: <https://docs.openstack.org/octavia/latest/>
11. OpenStack Foundation, “Telemetry (Ceilometer/Aodh) Documentation,” OpenStack Docs. [Online]. Available: <https://docs.openstack.org/telemetry/latest/>
12. OpenStack Foundation, “DevStack Documentation,” OpenStack Docs. [Online]. Available: <https://docs.openstack.org/devstack/latest/>

13. P. Mell and T. Grance, "The NIST Definition of Cloud Computing," National Institute of Standards and Technology, Special Publication 800-145, 2011.
14. R. Buyya, C. Vecchiola, and S. T. Selvi, Mastering Cloud Computing: Foundations and Applications Programming. Waltham, MA: Morgan Kaufmann, 2013.

APPENDICES

Appendix A – Command Reference and CLI Verification Screenshots

The following commands were used during implementation and verification of the OpenStack environment. Each command is listed with its purpose and expected outcome, followed by the actual terminal output captured for each verified command.

Command	Purpose / Expected Outcome
<code>source ~/devstack/devstack/ openrc admin admin</code>	Loads administrative credentials into the shell environment, enabling authenticated OpenStack CLI operations.
<code>openstack service list</code>	Lists all registered OpenStack services; used to confirm that core services (Keystone, Nova, Neutron, Glance, Cinder, Placement) are installed and registered.
<code>openstack compute service list</code>	Lists the status of Nova sub-services (scheduler, conductor, compute); used to confirm compute service health.
<code>openstack hypervisor list</code>	Lists registered hypervisors and their status; used to confirm QEMU hypervisor availability.
<code>openstack server list --all-projects</code>	Lists all virtual machine instances across all projects; used to confirm vm1's presence and ACTIVE status.
<code>openstack network list / subnet list / router list</code>	Lists registered virtual networks, subnets, and routers; used to confirm Neutron networking configuration.

```
divya_sree_m@LAPTOP-EDH45F4T:~/devstack/devstack$ openstack service list
```

ID	Name	Type
95a24b36e8b94d18af16cac186182ddb	cinder	block-storage
9e7533cc7edb40cf8cc36314d3bdf797	neutron	network
a8ba67056a0e46d9822cf6ee2f6615f2	glance	image
a9488ef42ff2487eb301bcde44629986	nova	compute
aa3815cbb9c44606ad018e071ca25ae3	placement	placement
ace30e626beb416793884b91e2e4e6d4	keystone	identity
fd0acb3f7c604443a2d077828b105d22	nova_legacy	compute_legacy

Figure A.1 – openstack service list – Terminal Output

```
divya_sree_m@LAPTOP-EDH45F4T:~/devstack/devstack$ openstack compute service list
```

ID	Binary	Host	Zone	Status	State	Updated At
6130d750-78d2-41b1-b064-22a2ebbc68eb	nova-scheduler	LAPTOP-EDH45F4T	internal	enabled	up	2026-08-20T05:28:47.000000
32a23a33-9871-4a00-ae94-2bc8f9caad0b	nova-conductor	LAPTOP-EDH45F4T	internal	enabled	up	2026-08-20T05:28:47.000000
74377916-8a31-4d0b-9da2-6529c5ae6cbf	nova-conductor	LAPTOP-EDH45F4T	internal	enabled	up	2026-08-20T05:29:11.000000
cb3e7d3b-0b1b-4ee4-98a6-30d51a6321de	nova-compute	LAPTOP-EDH45F4T	nova	enabled	up	2026-08-20T05:28:48.000000

Figure A.2 – openstack compute service list – Terminal Output

```
divya_sree_m@LAPTOP-EDH45F4T:~/devstack/devstack$ openstack hypervisor list
```

ID	Hypervisor Hostname	Hypervisor Type	Host IP	State
a78710d8-fb4d-43e7-b8a2-d6d1a8454a22	LAPTOP-EDH45F4T.localdomain	QEMU	172.21.30.246	up

Figure A.3 – openstack hypervisor list – Terminal Output

```
divya_sree_m@LAPTOP-EDH45F4T:~/devstack/devstack$ openstack server list --all-projects
```

ID	Name	Status	Networks	Image	Flavor	Project ID
c448dca4-7c8a-4878-8ee8-dfb11ded7838	vm1	ACTIVE	private=10.0.0.24, fdc5:8689:3cae:0:f816:3eff:fe7f:c240	N/A (booted from volume)	m1.tiny	3a38c3c2ff4b4d53a80a37653a047f6a

Figure A.4 – openstack server list --all-projects – Terminal Output


```
divya_sree_m@LAPTOP-EDH45F4T:~/devstack/devstack$ openstack network list
openstack subnet list
openstack router list
```

ID	Name	Subnets
853ec43d-13ae-44a0-983e-92958b024417	shared	fa3dd3bf-d218-4fb7-95ac-fb3235d74c3f
8ee45ca0-4bbb-4b17-b3fa-cf4cfeaa0cf1	public	2e2de05f-d2cb-4fe7-b11e-d901b4f9ed85, 660731f8-7c2d-4211-ba27-84e22e247f66
d603f001-84c7-414e-979d-ecb1279479d1	private	4ef8330d-0686-4fee-9626-21cd4fedfaef, b7803708-d316-45f7-a396-5a750fb9c821

Figure A.5 – openstack network/subnet/router list – Terminal Output

Appendix B – Horizon Dashboard Screenshots

The following screenshots were captured directly from the OpenStack Horizon web dashboard, showing the provisioned instance and networking configuration as viewed through the graphical interface rather than the CLI.

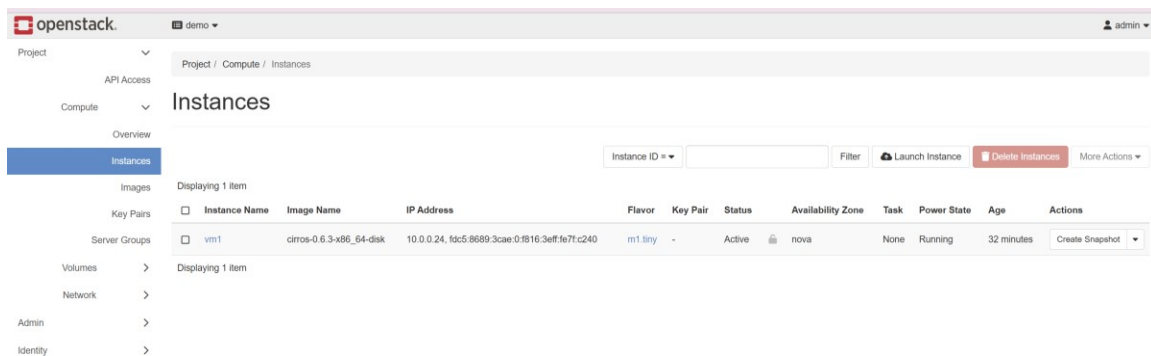


Figure B.1 – Horizon Dashboard – Instances Panel (Initial Verification)

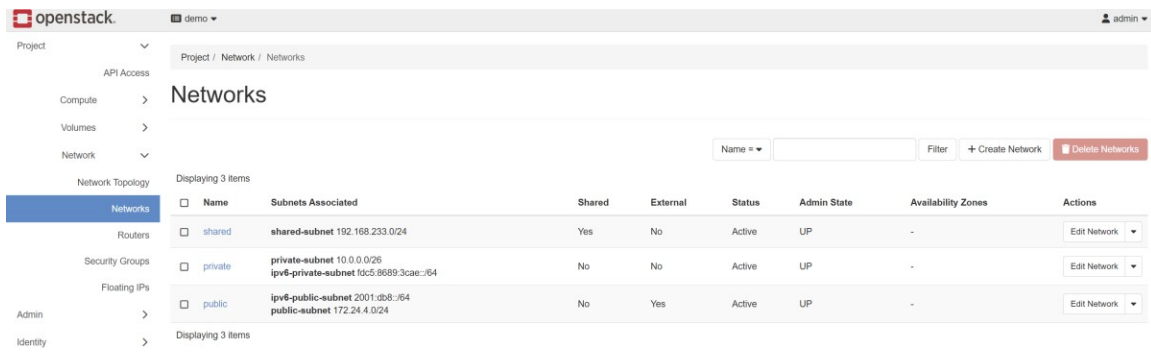


Figure B.2 – Horizon Dashboard – Networks Panel

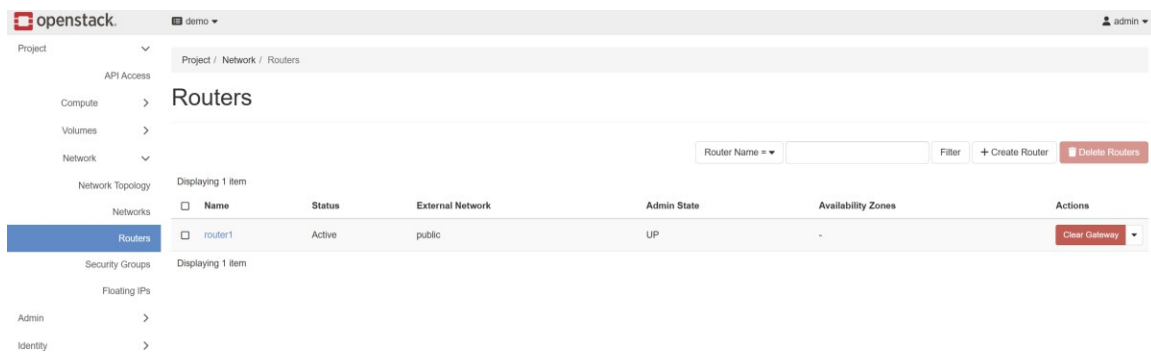


Figure B.3 – Horizon Dashboard – Routers Panel

Appendix C – Post-Recovery Verification Screenshot

The following screenshot was captured after the memory-exhaustion incident described in Section 4.9 was resolved. It shows the vm1 instance continuing to report an Active status well beyond the point of the initial fault, confirming that the instance and the underlying OpenStack services remained stable following recovery.

openstack.

demo

admin

Project

API Access

Compute

Overview

Instances

Images

Key Pairs

Server Groups

Volumes

Network

Admin

Identity

Project / Compute / Instances

Instances

Instance ID

Filter

Launch Instance

Delete Instances

More Actions

Displaying 1 item

	Instance Name	Image Name	IP Address	Flavor	Key Pair	Status	Availability Zone	Task	Power State	Age	Actions
☐	vm1	cirros-0.6.3-x86_64-disk	10.0.0.24, fdcd5.8689.3cae.0:f816:3eff:fe77:c240	m1.tiny	-	Active	nova	None	Running	13 hours, 11 minutes	Create Snapshot

Displaying 1 item

Figure C.1 – Horizon Dashboard – Instances Panel (Post-Recovery)